BankApp

New Generation Programming

Ege Türker- Ata Türker

User Login/Register

- The user can log in or, if they don’t have an account, they can register.

```
1. Login
2. Register
3. Quit
Choice:
```

```python
import sqlite3
from database import connect_db

def register_user(ad_soyad, email, sifre):
    conn = connect_db()
    if conn is None: return False

    cursor = conn.cursor()
    try:
        cursor.execute(
            "INSERT INTO users (ad_soyad, email, sifre) VALUES (?, ?, ?)",
            (ad_soyad, email, sifre)
        )
        user_id = cursor.lastrowid

        hesaplar = [('Vadesiz', 0.0), ('Yatirim', 0.0), ('Kredi', 0.0)]
        for tip, bakiye in hesaplar:
            cursor.execute(
                "INSERT INTO accounts (user_id, hesap_tipi, bakiye) VALUES (?, ?, ?)",
                (user_id, tip, bakiye)
            )

        conn.commit()
        print(f"\n✅ Registration successful! 3 accounts (Current, Investment, Loan) have been created.")
        return True
    except Exception as e:
        print(f"❌ Error: {e}")
        return False
    finally:
        conn.close()

def login_user(email, sifre):
    """
    Check email and password.
    Returns (user_id, full_name) if successful, otherwise None.
    """
    conn = connect_db()
    if conn is None: return None

    cursor = conn.cursor()
    cursor.execute(
        "SELECT user_id, ad_soyad FROM users WHERE email = ? AND sifre = ?",
        (email, sifre)
    )
    user = cursor.fetchone()
    conn.close()

    return user
```

Main Dashboard

- When the user logs in, they are redirected to the main dashboard, where they can select the main actions they want and navigate to their sub-sections.

```
=============================================
        MAIN DASHBOARD - WELCOME EGE TURKER
=============================================
1. 💳 Current Account Operations
2. 📈 Savings & Investment Operations
3. 🏦 Loan & Debt Operations
4. 📊 General Reports & Analysis
0. Secure Exit (Main Menu)
Select the account you want to operate: 
```

```python
def dashboard(user_id, ad_soyad):
    """First stop after login: Account Selection"""
    while True:
        print(f"\n=============================================")
        print(f"       MAIN DASHBOARD - WELCOME {ad_soyad.upper()}")
        print(f"=============================================")
        print("1. 💳 Current Account Operations")
        print("2. 📈 Savings & Investment Operations")
        print("3. 🏦 Loan & Debt Operations")
        print("4. 📊 General Reports & Analysis")
        print("0. Secure Exit (Main Menu)")

        hesap_secim = input("Select the account you want to operate: ")

        if hesap_secim == "1": vadesiz_menu(user_id, ad_soyad)
        elif hesap_secim == "2": yatirim_menu(user_id, ad_soyad)
        elif hesap_secim == "3": kredi_menu(user_id, ad_soyad)
        elif hesap_secim == "4": rapor_menu(user_id, ad_soyad)
        elif hesap_secim == "0": break
```

Current Account Operations

- Here, the user can check their balance, deposit money, withdraw money, send money to another user via email address, transfer money between their own accounts, view spending charts, review their transaction history, or return to the main menu.

```
--- 💳 CURRENT ACCOUNT | Ege Turker ---
1. Check Balance
2. Deposit Money
3. Withdraw Money (Categorized)
4. Send Money to Another User (Email)
5. Transfer Between My Accounts
6. Spending Analysis (Chart)
7. Recent Transactions
0. Back to Account Selection
Choice: 
```

```python
def vadesiz_menu(user_id, ad_soyad):
    while True:
        print(f"\n--- 💳 CURRENT ACCOUNT | {ad_soyad} ---")
        print("1. Check Balance")
        print("2. Deposit Money")
        print("3. Withdraw Money (Categorized)")
        print("4. Send Money to Another User (Email)")
        print("5. Transfer Between My Accounts")
        print("6. Spending Analysis (Chart)")
        print("7. Recent Transactions")
        print("0. Back to Account Selection")

        secim = input("Choice: ")
        if secim == "1": finance.hesaplari_listele(user_id)
        elif secim == "2":
            miktar = float(input("Amount: "))
            finance.para_ekle(user_id, "Vadesiz", miktar, "Income", "Deposit")
        elif secim == "3":
            miktar = float(input("Amount: "))
            kat = input("Category (Food/Bills/Entertainment/Transport/Other...): ")
            finance.para_cek(user_id, "Vadesiz", miktar, kat)
        elif secim == "4":
            mail = input("Recipient Email: ")
            miktar = float(input("Amount: "))
            finance.dis_transfer(user_id, mail, miktar)
        elif secim == "5":
            hedef = input("Target Account (Yatirim/Kredi): ")
            miktar = float(input("Amount: "))
            finance.ic_transfer(user_id, "Vadesiz", hedef, miktar)
        elif secim == "6": reports.grafik_analizi(user_id)
        elif secim == "7": reports.islem_gecmisi(user_id, 10)
        elif secim == "0": break
```

Saving/Investment Account

- In the Saving/Investment Account menu, the user can view the amount available for investment, calculate investment/returns, and transfer money from the investment account back to the main (demand deposit) account.

```
--- 📈 SAVINGS/INVESTMENT ACCOUNT | Ege Turker ---
1. Investment Balance
2. Calculate Interest/Return
3. Transfer Back to Current Account
0. Back to Account Selection
Choice:
```

```python
def yatirim_menu(user_id, ad_soyad):
    while True:
        print(f"\n--- 📈 SAVINGS/INVESTMENT ACCOUNT | {ad_soyad} ---")
        print("1. Investment Balance")
        print("2. Calculate Interest/Return")
        print("3. Transfer Back to Current Account")
        print("0. Back to Account Selection")

        secim = input("Choice: ")
        if secim == "1": finance.hesaplari_listele(user_id)
        elif secim == "2":
            oran = float(input("Annual Interest Rate (%): "))
            ay = int(input("Number of Months: "))
            finance.faiz_hesapla(user_id, oran, ay)
        elif secim == "3":
            miktar = float(input("Amount: "))
            finance.ic_transfer(user_id, "Yatirim", "Vadesiz", miktar)
        elif secim == "0": break
```

Loan and Debt Account

- Here, the user can view their debts, apply for a new loan, and perform a detailed debt analysis.

```
--- 🏦 LOAN ACCOUNT | Ege Turker ---
1. Loan Status
2. New Loan Application Simulation
3. Detailed Loan Analysis
0. Back to Account Selection
Choice:
```

```python
def kredi_menu(user_id, ad_soyad):
    while True:
        print(f"\n--- 🏦 LOAN ACCOUNT | {ad_soyad} ---")
        print("1. Loan Status")
        print("2. New Loan Application Simulation")
        print("3. Detailed Loan Analysis")
        print("0. Back to Account Selection")

        secim = input("Choice: ")
        if secim == "1":
            finance.hesaplari_listele(user_id)
        elif secim == "2":
            tutar = float(input("Requested Amount: "))
            ay = int(input("Repayment Period (Months): "))
            finance.kredi_basvurusu(user_id, tutar, ay)
        elif secim == "3":
            reports.kredi_analizi(user_id)
        elif secim == "0": break
```

Reports and Analysis

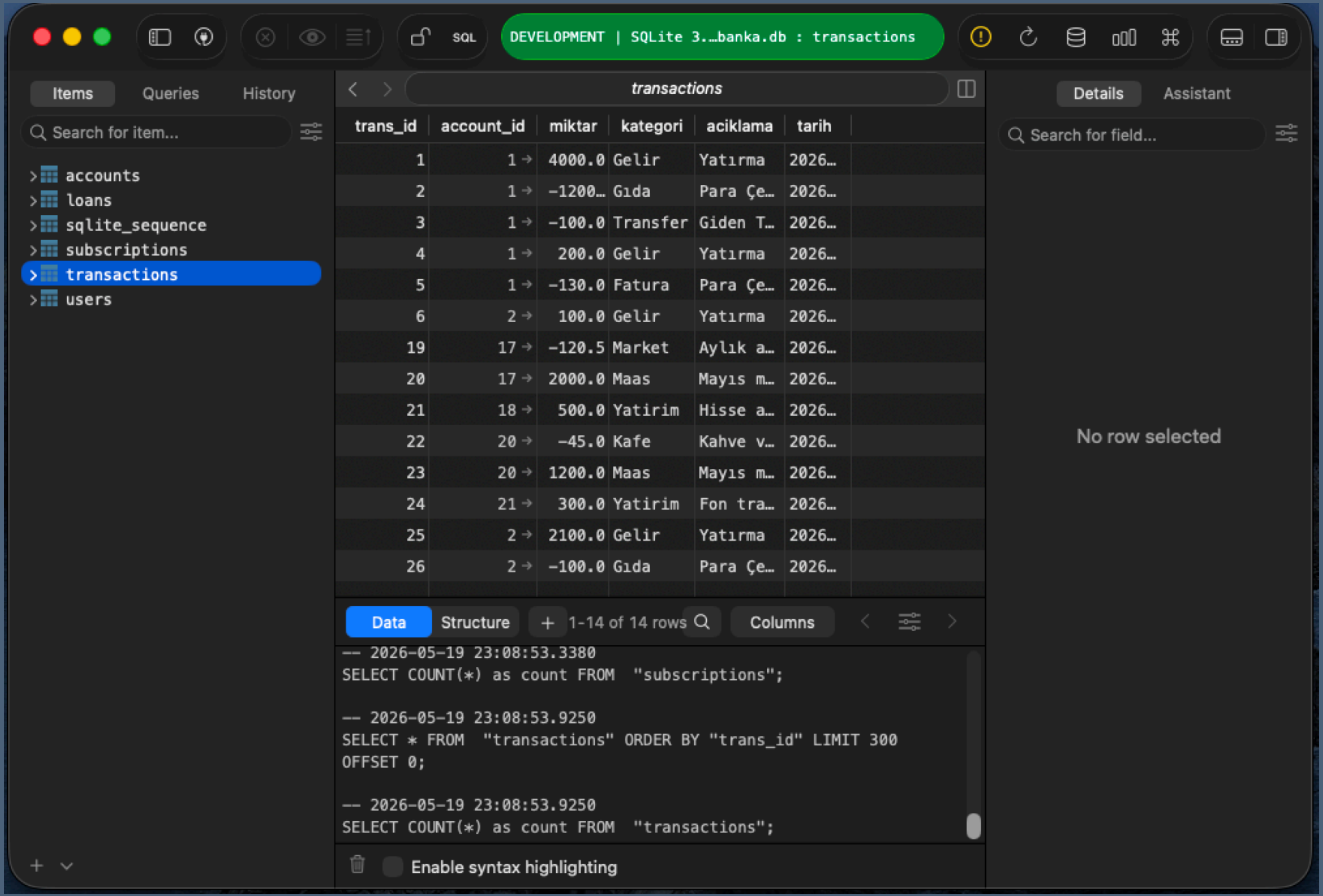
- The user can perform spending analysis, where the program displays a chart in front of them. They can monitor their monthly expenses, analyze their debts, view account summaries, and review past transactions.

```
--- 📊 REPORTS & ANALYSIS | Ege Turker ---
1. Spending Analysis (Chart)
2. Monthly Summary (Income-Expense)
3. Loan Analysis
4. Account Summary
5. Recent Transactions
0. Back
Choice:
```

```python
def rapor_menu(user_id, ad_soyad):
    """General Reports and Analysis Menu"""
    while True:
        print(f"\n--- 📊 REPORTS & ANALYSIS | {ad_soyad} ---")
        print("1. Spending Analysis (Chart)")
        print("2. Monthly Summary (Income-Expense)")
        print("3. Loan Analysis")
        print("4. Account Summary")
        print("5. Recent Transactions")
        print("0. Back")

        secim = input("Choice: ")
        if secim == "1": reports.grafik_analizi(user_id)
        elif secim == "2": reports.aylik_ozet(user_id)
        elif secim == "3": reports.kredi_analizi(user_id)
        elif secim == "4": reports.hesap_ozeti(user_id)
        elif secim == "5":
            n = int(input("How many transactions would you like to see? "))
            reports.islem_gecmisi(user_id, n)
        elif secim == "0": break
```

Database and Tables



Statistical Data

Matplotlib

- We enhanced the statistical parts of the project using Matplotlib.

```python
import sqlite3
import matplotlib.pyplot as plt
from database import connect_db
from datetime import datetime, timedelta

def grafik_analizi(user_id):
    """Spending analysis by category with a simple chart."""
    conn = connect_db()
    cursor = conn.cursor()
    try:
        cursor.execute("""
            SELECT kategori, COUNT(*) as adet, ABS(SUM(miktar)) as toplam
            FROM transactions t
            JOIN accounts a ON t.account_id = a.account_id
            WHERE a.user_id = ? AND a.hesap_tipi = 'Vadesiz' AND t.miktar < 0
            GROUP BY kategori
            ORDER BY toplam DESC
        """, (user_id,))

        veriler = cursor.fetchall()
        if not veriler:
            print("\n❌ No spending data found.")
            return

        toplam_harcama = sum(item[2] for item in veriler)

        kategoriler = [item[0] for item in veriler]
        toplamlar = [item[2] for item in veriler]

        print("\n--- 📊 SPENDING ANALYSIS (Recent Transactions) ---")
        print(f"Total Spending: {toplam_harcama:.2f} TL\n")

        for kategori, adet, toplam in veriler:
            yuzde = (toplam / toplam_harcama) * 100
            print(f"{kategori:15} {adet:>3} transactions | {yuzde:5.1f}% ({toplam:.2f} TL)")

        plt.figure(figsize=(8, 6))
        plt.bar(kategoriler, toplamlar, color="tab:blue")
        plt.title("Spending Distribution")
        plt.xlabel("Category")
        plt.ylabel("Total Spending (TL)")
        plt.xticks(rotation=25, ha="right")
        plt.tight_layout()
        plt.show()

    except Exception as e:
        print(f"❌ Error: {e}")
    finally:
        conn.close()
```

Project Infrastructure

- Since the project becomes more complex as it develops, we paid special attention to the file structure. We separated user authentication, graph generation, and analysis into different files, thus creating a modular structure that is open to further development.